



## **Ingeniería de sistemas**

### **Docente**

Leidy Mariani Dorado Flórez

### **Materia**

Sistemas distribuidos

### **Título**

Sistemas Distribuidos – Diseño e Integración con Docker Compose

### **Presentado por**

Nasly Stefania Collazos

**Popayán, Cauca, 17 de Marzo de 2026**

## 1. Objetivo

Diseñar e implementar un sistema distribuido básico utilizando Docker Compose, comprendiendo la relación entre servicios y su comunicación.

## 2. Fase de diseño del sistema

### 1. Definir el tipo de sistema (ejemplo: e-commerce, app de domicilios, plataforma educativa)

R: Aplicación de domicilios, donde los usuarios pueden pedir productos y recibirlos en casa.

### 2. Identificar mínimo tres servicios (frontend, backend, base de datos)

R:

- **Servicio de pedidos:** Se encarga de gestionar los pedidos (crear, consultar, actualizar estado).
- **Servicio de usuarios:** Maneja el registro, inicio de sesión y datos de los usuarios.
- **Servicio de notificaciones:** Envía avisos al usuario (pedido confirmado, en camino, entregado)

### 3. Describir cómo se comunican

- El servicio de usuarios valida la información del cliente
- El servicio de pedidos usa los datos del usuario para crear pedidos.
- El servicio de notificaciones recibe eventos del servicio de pedidos y envía mensajes al usuario

Ejemplo:

1. Usuario hace pedido
2. Servicio de pedidos lo registra
3. Servicio de notificaciones envía confirmación

### 4. Indicar qué ocurre si un servicio falla.

- Si falla usuarios, no se podría validar ni iniciar sesión
- Si falla pedidos, no se pueden hacer pedidos
- Si falla notificaciones, los pedidos funcionan, pero no llegan los avisos

## 4. Análisis

### 1 ¿Cuál es el rol de cada servicio?

- **Servicio de pedidos:** Se encarga de gestionar los pedidos (crear, consultar, actualizar estado).
- **Servicio de usuarios:** Maneja el registro, inicio de sesión y datos de los usuarios.
- **Servicio de notificaciones:** Envía avisos al usuario (pedido confirmado, en camino, entregado)

### 2 ¿Qué ventajas tiene dividir el sistema?

**R:** Dividir el sistema en varios servicios permite:

- Mayor organización ya que cada servicio cumple una función específica.
- Mayor escalabilidad, porque se pueden aumentar recursos solo en el servicio necesario.
- Mantenimiento sencillo, al poder modificar un servicio sin afectar a los demás.
- Aislamiento de fallos, ya que si un servicio falla, los otros podrán seguir funcionando.

### 3 ¿Cómo se comunican los contenedores?

**R:** Los contenedores se comunican a través de la red interna de docker compose

- Cada servicio puede acceder a otro usando su nombre como host (por ejemplo: db, users-service).
- La comunicación se realiza mediante protocolos como HTTP (entre frontend y backend) o conexiones a base de datos.
- No es necesario configurar redes manualmente, ya que docker compose las crea automáticamente.

## 5. Exploración para la siguiente clase

### 1 Investigar cómo definir variables de entorno

**R:** Las variables de entorno se configuran dentro del archivo `docker-compose.yml` utilizando la sección `environment` en cada servicio. En el caso del backend, estas variables permiten enviar las credenciales de la base de datos MySQL sin necesidad de incluirlas directamente en el código fuente.

Por ejemplo, se definen valores como el `host`, `usuario`, `contraseña` y `nombre` de la base de datos. Posteriormente, en la aplicación desarrollada con Node.js, dichas variables se consultan mediante `process.env`, lo que facilita su uso de forma segura y dinámica.

### 2 Buscar cómo conectar un backend con una base de datos

**R:** El backend establece la conexión con MySQL mediante la librería `mysql2/promise`. Para ello, se configura un pool de conexiones utilizando un objeto de configuración que obtiene sus valores desde las variables de entorno.

Cuando el servidor se inicia, se ejecuta una función encargada de inicializar la base de datos, la cual se conecta al servicio y crea las tablas necesarias en caso de que no existan. En situaciones donde MySQL aún no está disponible, el sistema implementa un mecanismo de reintento automático cada cinco segundos hasta lograr la conexión.

### 3 Identificar posibles errores de conexión

**R:** Durante la conexión entre el backend y la base de datos pueden presentarse distintos errores, principalmente relacionados con la disponibilidad del servicio, la configuración de credenciales o problemas de red.}

- Uno de los errores más comunes es **ECONNREFUSED**, que ocurre cuando MySQL aún no ha terminado de iniciar en el contenedor y el backend intenta conectarse antes de tiempo. Para mitigar este problema, el sistema implementa un mecanismo de reintento automático cada cinco segundos.
- También puede presentarse el error **ER\_ACCESS\_DENIED\_ERROR**, el cual indica que las credenciales (`usuario` o `contraseña`) definidas en las variables de entorno son incorrectas. En este caso, es necesario verificar que los datos coincidan con la configuración de la base de datos.
- Otro posible fallo es **ER\_BAD\_DB\_ERROR**, que sucede cuando la base de datos especificada no existe. Esto puede resolverse asegurando su correcta creación o validando el nombre definido en las variables de entorno.
- Finalmente, el error **ETIMEDOUT** se presenta cuando el backend no logra establecer comunicación con el contenedor de MySQL, generalmente debido a problemas en la red de Docker o en la configuración de los servicios.

En conjunto, estos errores pueden ser gestionados mediante buenas prácticas como la correcta configuración de variables de entorno, la validación de servicios en Docker y el uso de reintentos automáticos para garantizar la conexión.